Density-Based Classification of Lighting Conditions on a Memory-Constrained ESP32: A Proof of Concept

Gianluca D'Altorio
Department of Computer Engineering Technology
Vanier College

I. Proposed Pipeline

This section describes the part of the pipeline that runs on the device itself. Everything else, grouping the training data into classes, building a density map for each class, and shrinking those maps into small byte tables, happens beforehand on a regular computer. The device only ever sees the finished byte tables, never the original training data.

Algorithm 1 describes what the device does when it receives a new point. First, the point is rescaled with the same bounds used during training (for the IR channel, after the same $\log(1+x)$ transform used in training, see Section III). The scaled point selects one cell of the grid. The device reads one stored byte from that cell for each class and picks the class with the highest value, unless even that value falls below a reject threshold τ, in which case it reports UNKNOWN instead of guessing. Without this last step the device would confidently name a class in places where it has never seen any data.

Since the whole process is a few reads and comparisons, its cost does not depend on how much data was used to build the tables. Nearest cell lookup (line 5 of Algorithm 1) is used rather than a bilinear blend of the four surrounding cells: it costs one byte read per class instead of four, and on this dataset the classes are separated widely enough that the extra smoothness is not needed.

II. Application

The application is an ESP32 that must recognize the lighting condition it is in, using two sensors: a UV sensor and an IR channel. Such a device has a few kilobytes of flash to spare next to its radio and sleep logic, so a classifier that costs about one kilobyte, rather than a neural network or a library of several kilobytes, directly leaves room for the rest of the firmware.

Labeled data is cheap to collect in this particular case, which lets us check the pipeline against ground truth, but the pipeline itself never uses the labels: classes are discovered by clustering. This matters because the shapes in a sensor feature space are not round or evenly spaced. In our data each indoor class is a thin streak along the IR axis (indoor UV reads exactly zero), and sunlight sits far away from all of them, so a center-based method such as k-means would have to place centers on streaks of very different lengths. A density-based method follows the actual shape of each class.

Algorithm 1 Table-lookup classification (online path).

Input: raw query (u, r) with r the IR count; $r \leftarrow \log(1 + r)$
Input: One byte table Q_c per class, each $G \times G$, values 0–255
Input: The scaling bounds $(x_{\min}, x_{\max})$, $(y_{\min}, y_{\max})$ used in training
Input: The grid size G and a reject threshold τ
Output: A predicted class $\hat{c}$, or UNKNOWN

1: $\tilde{x} \leftarrow \dfrac{x - x_{\min}}{x_{\max} - x_{\min}}, \quad \tilde{y} \leftarrow \dfrac{y - y_{\min}}{y_{\max} - y_{\min}}$
2: col $\leftarrow \lfloor \tilde{x}\, G \rfloor$, clipped to $[0, G-1]$
3: row $\leftarrow \lfloor \tilde{y}\, G \rfloor$, clipped to $[0, G-1]$
4: **for** each class c **do**
5: $\quad v_c \leftarrow Q_c[\text{row}, \text{col}]/255$ ▷ one byte read
6: **end for**
7: $\hat{c} \leftarrow \arg\max_c v_c$ ▷ the winning class
8: **if** $v_{\hat{c}} < \tau$ **then**
9: $\quad$ **return** UNKNOWN ▷ outside every learned region
10: **end if**
11: **return** $\hat{c}$

A. Proof of Concept

Figure 1 shows the working prototype: an ESP32 development board with a GUVA-S12SD analog UV sensor on GPIO34 and a TSL2591 light sensor on I^2C (SDA on pin 21, SCL on pin 22), of which only the infrared channel is used. The device classifies live and prints the predicted class over serial. A short video of the device switching between sample conditions has been attached to the assignment submission.

III. Method and Setup

A. Hardware and Data Collection

The UV signal comes from a GUVA-S12SD read on the ESP32's 12-bit ADC (0–4095). The IR signal comes from the TSL2591's IR channel over I^2C. Both channels are smoothed on the device with an exponential moving average, $y_t = \alpha x_t + (1-\alpha)\, y_{t-1}$ with $\alpha = 0.2$. This gives a time constant of about 0.55 s at the effective rate of about 8.2 Hz (one sample per 122 ms).

Four lighting conditions were recorded, one labeled CSV each: LED, fluorescent, shade and sun, with 600 samples per class (2 400 in total), recorded on October 7, with columns

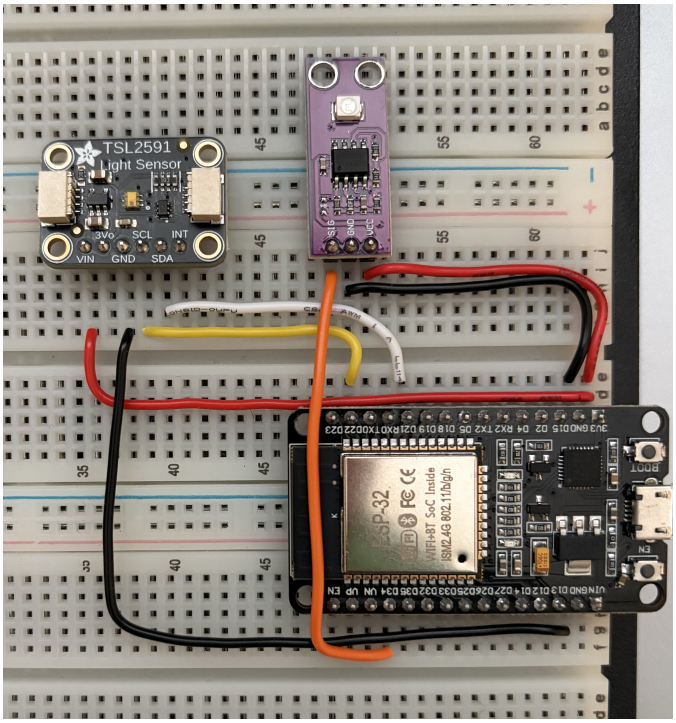


Fig. 1. The proof-of-concept board: ESP32, GUVA-S12SD UV sensor and TSL2591 IR sensor.

`uv_sensor`, `IR_sensor` and `label`. The labels are used only to evaluate the model, never to build it.

B. Preprocessing and Splits

Indoor UV is exactly zero, while IR counts span more than two orders of magnitude (130–390 indoors, about 28 000 in sunlight). Scaling the raw IR to $[0, 1]$ would squeeze all three indoor classes into a sliver, so IR is transformed with $\log(1 + x)$ before scaling. Both features are then scaled to $[0, 1]$ using the minimum and maximum of the training split alone, and the same bounds are shipped to the device. The data was split 60/20/20 into 1 440 training samples, 480 validation and 480 test samples, stratified by label, with a fixed random seed. Since the samples are consecutive readings of a smoothed signal, neighboring samples are near-duplicates, so these random splits are optimistic; the separately recorded batches described in Section IV are the real test.

C. Model Construction

Classes were discovered with DBSCAN on the scaled training split with $\varepsilon = 0.02$ and a minimum of 8 points per neighborhood. This gave four clusters of 360 points each and no noise points. $\varepsilon = 0.06$ gave the same four clusters, so the result is not sensitive to the exact value in that range and the smaller, tighter $\varepsilon = 0.02$ was kept; $\varepsilon = 0.30$ merged the three indoor classes into one cluster of 1 080 points. Conversely, a radius that is too small fragments classes: $\varepsilon = 0.005$ produced six clusters and seven noise points, with three classes intact (360 points each) but the fourth split into clusters of 306, 27 and 20 points. At that radius the neighborhoods no longer overlap enough to connect every point of a class, so sparser stretches of the same class break off as separate clusters or noise. A Gaussian kernel density estimate was fitted to each cluster, peak-normalized to 1.0, evaluated on a $G \times G$ grid and quantized to one byte per cell. The reject threshold is $\tau = 0.12$.

The bandwidth h and grid size G were chosen by an end-to-end sweep on the validation split: for each pair, the stored byte tables were built and used to classify the validation points, and the fraction classified correctly and the fraction wrongly rejected were recorded. Choosing h by held-out log-likelihood, as in the earlier synthetic experiment, failed on this data: it always picked the smallest h because the UV axis has zero variance and near-duplicate neighbors leak across the split. With $G = 16$, every $h \geq 0.015$ gave 100% correct and 0% rejected, while smaller values wrongly rejected 0.2–0.8% of points. Both $h = 0.02$ and $G = 16$ were frozen.

D. Deployment and Measurement

A generator script writes `model.h` containing the four 16×16 byte tables (1 024 bytes in flash), the scaling bounds (16 bytes), the class names and τ. The firmware reads the sensors, applies the same smoothing filter and $\log(1 + x)$, scales, indexes the tables and prints `idx, uv, ir, row, col, class, latency_us` over serial.

Cost was measured by differencing. A copy of the firmware in which the classifier does not read the table (it uses a constant instead) was compiled alongside the full firmware, and the compiler's reported program size and global-variable size were subtracted. Lookup latency was timed with `micros()` around the classification call.

IV. Evidence

The evidence follows the structure of the data: the model is built on the training split, tuned on validation, and then compared with ground truth, with the desktop model, and with the real hardware.

A. Classes are Discoverable Without Labels

Table I compares DBSCAN's clusters with the ground-truth labels. Each cluster is named by the majority label of its members. DBSCAN recovered all four lighting conditions: every one of the 1,440 training samples fell in the cluster matching its label, with no noise points, without ever seeing the labels. This is a clean result partly because the classes are well separated in this feature space; it does not show that clustering would stay this clean on conditions that overlap. Figure 2 shows the clusters.

TABLE I
DBSCAN clusters (columns) versus ground-truth labels (rows), all 1 440 training samples.

True label	Cluster 0	Cluster 1	Cluster 2	Cluster 3
LED	360	0	0	0
fluorescent	0	360	0	0
shade	0	0	360	0
sun	0	0	0	360

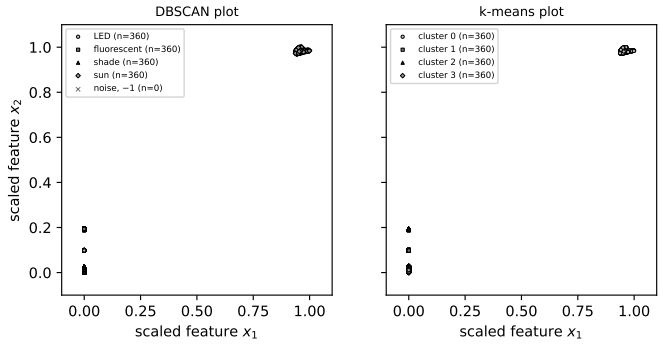


Fig. 2. DBSCAN recovers the four lighting conditions from unlabeled data (scaled UV and log-scaled IR). Clusters are named by their majority ground-truth label.

B. The Density Maps are Well-Formed and Survive Storage

Figure 3 shows the per-class density maps and Figure 4 the combined decision surface with the reject threshold. Figure 5 shows the same maps stored as one byte per cell on the 16×16 grid. Quantizing each value to one of 256 levels added a measured worst-case error of 0.177% of a class's peak, within the theoretical bound of 0.196%. The three indoor classes occupy adjacent bands along the IR axis. These bands follow from the scaling bounds stored in `model.h` and the 16-row grid: the log-scaled IR range (4.885 to 10.370) is divided into 16 equal rows, and converting the row edges back to counts gives boundaries near 185, 262 and 520. Reading the non-zero cells of the stored tables, shade covers row 0 (IR below about 185 counts), LED row 1 (about 185–262) and fluorescent rows 2–3 (about 262–520). The classes are therefore brightness bands rather than source types, which matters for the limitations in Section V.

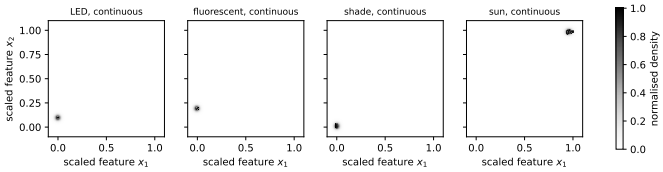


Fig. 3. Per-class density maps from kernel density estimation ($h = 0.02$), each normalized to its own peak.

C. What the Model Costs: Predicted

Table II lists the flash cost of storing the maps at several precisions for $K = 4$ classes and $G = 16$. Storing as `uint8` predicts 1 024 bytes of tables plus 16 bytes of bounds, an eightfold reduction versus float64.

D. Filtering

Figure 6 plots the raw and filtered IR signal for the LED and fluorescent classes, with the class boundaries drawn. Both signals contain brief, sharp downward glitches, four in each plot, where the raw reading drops to between roughly 40 and 130 counts for a sample or two. Unfiltered, every one of these

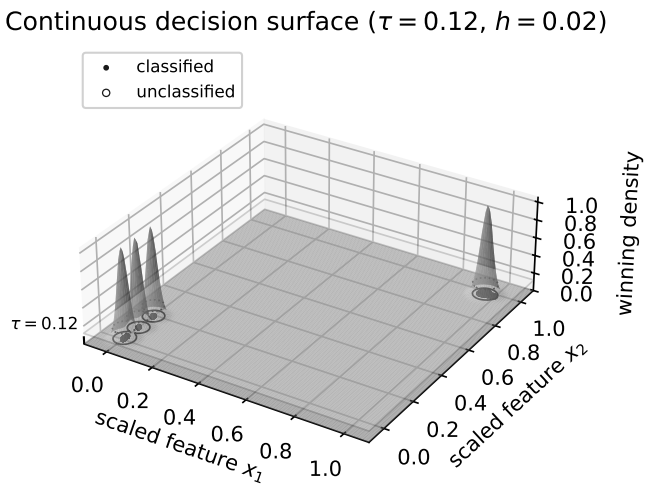


Fig. 4. Combined decision surface: at every location the largest of the four normalized class densities, with the reject threshold $\tau = 0.12$.

would cross a class boundary: an LED or fluorescent glitch would read as shade. The exponential moving average shrinks them. In the fluorescent recording all four filtered dips stay above the LED boundary (about 262 counts), so no glitch changes the class. In the LED recording three of four stay inside the LED band, but the largest dips to about 155 counts, below the shade boundary (about 185), so the filter reduces but does not remove misclassification. The price of the filter is lag: a step change takes about 1.4 s to settle (about 2.5 time constants).

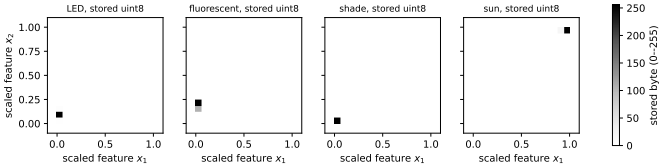


Fig. 5. The same four maps stored as `uint8` on the 16×16 grid.

E. A Query Classified, and a Query Rejected

A reading with a filtered IR value in the middle of a class band lands in a single grid cell, and reading that cell's four stored bytes gives one clear winner. The reject mechanism is shown by overcast sunlight, which was not part of the training data. Its readings fall in a cell where all four bytes are zero, so the device reports UNKNOWN instead of choosing the nearest class. This is the behavior the threshold is for: "near data" is not "is data".

F. On-Device Versus Desktop Agreement

A fresh batch of 581 samples per class (2 324 in total) was logged from the running device on October 12, five days after the training recordings, at the same location, and replayed through the desktop Python model using the same scaling

and tables. The device and the desktop model disagreed on none of the 2 324 samples, which confirms that the on-device arithmetic (single-precision `log1pf`, scaling and indexing) reproduces the desktop decisions.

TABLE II
MODEL FOOTPRINT BY STORAGE FORMAT ($K = 4$ CLASSES, $G = 16$).

Storage	Bytes/cell	Per class	All K classes
uint8	1	256	1 024
uint16	2	512	2 048
float32	4	1 024	4 096
float64	8	2 048	8 192

Against ground truth (Table III), LED, fluorescent and shade were each classified correctly on 100% of their 581 samples. Sun was classified correctly on 0%: all 581 samples were UNKNOWN, because this batch was recorded in overcast conditions that differ from the clear-sun training data. The device and desktop agreed on this too, so this is a property of the model, not a porting error. It is discussed in Section V.

G. Measured Versus Predicted Cost

Table IV compares prediction and measurement. Flash grew by 1 060 bytes (289 133 bytes with the table lookup versus 288 073 bytes without it), against 1 024 predicted for the tables alone. The extra 36 bytes are the other things the model brings with it that the prediction did not count: the 16 bytes of scaling bounds, and the class names, threshold and lookup code (about 20 bytes). Global variables (SRAM) were identical, 21 944 bytes in both builds, so the tables add no SRAM because they stay in flash. Lookup latency was 4–8 µs, negligible next to the 122 ms sample period.

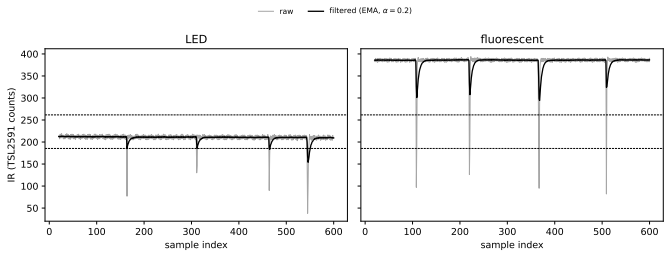


Fig. 6. Raw and filtered IR for LED and fluorescent lighting, with class boundaries.

V. LIMITATIONS AND REMAINING UNCERTAINTY

The evidence supports the pipeline for the four conditions tested, but several limits bound how far it can be trusted.

Conditions outside the training data are rejected, not recognized. The fresh sun batch was overcast, unlike the clear-sun training data, and was reported as UNKNOWN on every sample (100%). This is the reject mechanism working, but it also shows the tables are sparse: any in-between condition lands in an empty cell. Covering this would require adding an overcast class and retraining with a new held-out batch, which was not within the scope of this particular experiment.

The classes are brightness bands, not source types. Shade, LED and fluorescent differ mainly in IR level, so distance and position matter as much as the source. Light through blinds was classified as LED, and covering the IR sensor, which gives IR below the training minimum, was clipped to the edge row and classified as shade. The fluorescent level also drifted by over 100 counts between recordings and sat barely above the LED boundary (minimum 261.65 against a boundary at 261.63), so a small shift would change its class.

TABLE III
FRESH-BATCH RESULTS (581 SAMPLES PER CLASS).

Class	Device = desktop	Correct vs truth	UNKNOWN
LED	100%	100%	0%
fluorescent	100%	100%	0%
shade	100%	100%	0%
sun	100%	0%	100%

Filter lag and glitches. The exponential moving average needs about 1.4 s to follow a step change. An isolated LED glitch crossed into the shade band even after filtering.

Optimistic validation. Consecutive readings are near-duplicates, so random splits of one recording overstate accuracy and likelihood-based tuning failed (Section III). The 100% figures on validation and test splits should not be read as expected field accuracy; only the separately recorded batches are independent, and they cover one home, with training data from October 7 and the fresh batch from October 12.

TABLE IV
PREDICTED VERSUS MEASURED COST OF THE STORED MODEL.

Quantity	Predicted	Measured
Flash (bytes)	1 024	1 060
SRAM (bytes)	0	0
Latency (µs)	—	4–8

Agreement is not accuracy. The on-device-versus-desktop result shows the stored model is implemented faithfully, not that its decisions are right. Ground-truth accuracy comes from Table I and the held-out batch.

Choices were tuned to this dataset. h, G, ε and $\tau = 0.12$ were chosen for this two-feature dataset; τ was fixed rather than optimized. What to measure next: sweep τ against a labeled set that includes intermediate conditions, and repeat the evaluation in other rooms and weather.